



Diff, Stash and Tags

This guide will help you understand the different commands related to diff, tags and stash in git. These are not main stream commands but they are very useful in certain situations.

Git diff

The `git diff` is an informative command that shows the differences between two commits. It is used to compare the changes made in one commit with the changes made in another commit. Git consider the changed versions of same file as two different files. Then it gives names to these two files and shows the differences between them.

How to Read the Diff Output

- `a/` – the original file (before changes)
- `b/` – the updated file (after changes)
- `---` – marks the beginning of the original file
- `+++` – marks the beginning of the updated file
- `@@` – shows the line numbers and position of changes

Here the file A and file B are the same file but different versions.

Git will show you the changes made in the file A and file B. It will also show you the line number where the change occurred along with little preview of the change.

Comparing Working Directory and Staging Area

```
git diff
```

This command shows the unstaged changes in your working directory compared to the staging area. This command alone will not show you the changes made in the file A and file B, you need

to provide options to show the changes.

Comparing Staging Area with Repository

```
git diff --staged
```

This command shows the changes between your last commit and the staging area (i.e., changes that are staged and ready to be committed).

Comparing Two Branches

```
git diff <branch-name-one> <branch-name-two>
```

This command compares the difference between two branches.

Another way to compare the difference between two branches is to use the following command:

```
git diff branch-name-one..branch-name-two
```

Comparing Specific Commits:

```
git diff <commit-hash-one> <commit-hash-two>
```

This command compares the difference between two commits.

Git Stash

Stash is a way to save your changes in a temporary location. It's useful when switching branches without losing work. You can then come back to the file later and apply the changes.

Conflicting changes will not allow you to switch branches without committing the changes. Another alternative is to use the `git stash` command to save your changes in a temporary location.

```
git stash
```

This command saves your changes in a temporary location. It is like a stack of changes that you can access later.

Naming the stash

You can also name the stash by using the following command:

```
git stash save "work in progress on X feature"
```

View the stash list

You can view the list of stashes by using the following command:

```
git stash list
```

Apply the Most Recent Stash

You can apply the stash by using the following command:

```
git stash apply
```

Apply Specific Stash

You can apply the specific stash by using the following command:

```
git stash apply stash@{0}
```

Here `stash@{0}` is the name of the stash. You can use the `git stash list` command to get the name of the stash.

Applying and Drop a Stash

You can apply and drop the stash by using the following command:

```
git stash pop
```

This command applies the stash and drops it from the stash list.

Drop the stash

You can drop the stash by using the following command:

```
git stash drop
```

Applying stash to a specific branch

You can apply the stash to a specific branch by using the following command:

```
git stash apply stash@{0} <branch-name>
```

Clearing the stash

You can clear the stash by using the following command:

```
git stash clear
```

Git Tags

Tags are a way to mark a specific point in your repository. They are useful when you want to remember a specific version of your code or when you want to refer to a specific commit. Tags are like sticky notes that you can attach to your commits.

Creating a tag

You can create a tag using the following command:

```
git tag <tag-name>
```

This command creates a new tag with the specified name. The tag will be attached to the current commit.

Create an annotated tag

You can create an annotated tag using the following command:

```
git tag -a <tag-name> -m "Release 1.0"
```

This command creates an annotated tag with the specified name and message. The tag will be attached to the current commit.

List all tags

You can list all tags using the following command:

```
git tag
```

This command lists all the tags in your repository.

Tagging a specific commit

You can tag a specific commit using the following command:

```
git tag <tag-name> <commit-hash>
```

Push tags to remote repository

You can push tags to a remote repository using the following command:

```
git push origin <tag-name>
```

Delete a tag

You can delete a tag using the following command:

```
git tag -d <tag-name>
```

Delete tag on remote repository

You can delete a tag on a remote repository using the following command:

```
git push origin :<tag-name>
```

Conclusion

In this section, we explored how to use Git's diff, stash, and tags commands. Though not used as frequently as add, commit, or push, they are incredibly helpful in debugging, context switching, and release management. See you next tutorial.

Start your journey with ChaiCode

All of our courses are available on chaicode.com. Feel free to check them out.

♥ Compliled by: Hitesh Choudhary

Last updated: Apr 14, 2025

← Previous

Next →

Branches in Git

Managing History



Contribute



Community



Sponsor